

Summary

This C program creates a very simple Layer-2 tunnel between a Linux TUN interface and a raw Ethernet socket using a custom EtherType (0x88B5).

It operates in two modes:

- **TX mode (tx)**
Reads IP packets from a TUN interface, wraps them in an Ethernet frame with a custom EtherType, and sends them over a physical Ethernet interface.
- **RX mode (rx)**
Receives Ethernet frames with the custom EtherType from the Ethernet interface, strips the Ethernet header, and injects the IP packet into the TUN interface.

The result is effectively a lightweight custom Ethernet encapsulation mechanism for IP traffic.

High-Level Flow

TX Path

```
graph TD
    A["[TUN interface]"] --> B["↓"]
    B --> C["Read raw IP packet"]
    C --> D["↓"]
    D --> E["Add Ethernet header"]
    E --> F["↓"]
    F --> G["Send via raw socket"]
    G --> H["↓"]
    H --> I["[Physical NIC]"]
```

RX Path

```
graph TD
    A["[Physical NIC]"] --> B["↓"]
    B --> C["Receive Ethernet frame"]
    C --> D["↓"]
    D --> E["Check custom EtherType"]
    E --> F["↓"]
    F --> G["Remove Ethernet header"]
    G --> H["↓"]
    H --> I["Write IP packet into TUN"]
    I --> J["↓"]
    J --> K["[TUN interface]"]
```

Detailed Function Commentary

Headers and Constants

```
#define CUSTOM_ETHERTYPE 0x88B5
```

```
#define BUF_SIZE 2000
```

Purpose

- CUSTOM_ETHERTYPE
 - Defines the custom Ethernet protocol identifier.
 - Frames sent by this program use EtherType 0x88B5.
- BUF_SIZE
 - Maximum packet/frame buffer size.

Notes

- 0x88B5 is in the experimental/local EtherType range.
 - 2000 bytes is enough for standard MTU Ethernet frames.
-

tun_alloc()

```
static int tun_alloc(char *dev)
```

Purpose

Creates and configures a Linux TUN device.

What It Does

1. Opens /dev/net/tun
2. Requests:
 - IFF_TUN
 - TUN mode (IP packets only)
 - IFF_NO_PI
 - No extra packet metadata header
3. Assigns interface name
4. Returns file descriptor for packet I/O

Important Detail

TUN devices operate at Layer 3:

- Reads/writes IP packets only
- No Ethernet headers

Example

If interface is tun0:

ip tuntap add dev tun0 mode tun

Equivalent behavior is achieved programmatically here.

raw_socket_open()

```
static int raw_socket_open(char *iface)
```

Purpose

Creates a raw Ethernet socket bound to a physical interface.

What It Does

1. Creates:

```
socket(AF_PACKET, SOCK_RAW, htons(CUSTOM_ETHERTYPE))
```

This allows direct Ethernet frame access.

2. Retrieves interface index using:

```
SIOCGIFINDEX
```

3. Binds socket to:

- specific NIC
- specific EtherType

Result

The socket will:

- send raw Ethernet frames
 - receive only frames with EtherType 0x88B5
-

get_mac()

```
static void get_mac(int fd, char *iface, unsigned char *mac)
```

Purpose

Obtains MAC address of the Ethernet interface.

Mechanism

Uses:

```
ioctl(... SIOCGIFHWADDR ...)
```

Then copies:

```
ifr.ifr_hwaddr.sa_data
```

into the supplied buffer.

tx_loop()

static void tx_loop(...)

Purpose

Encapsulates TUN IP packets inside Ethernet frames.

TX Processing Steps

1. Read packet from TUN

read(tun_fd, buf, sizeof(buf))

This yields a raw IP packet.

2. Construct Ethernet header

struct ether_header *eth

Sets:

- destination MAC
 - source MAC
 - custom EtherType
-

3. Append IP payload

memcpy(frame + sizeof(struct ether_header), buf, n);

Resulting frame:

```
+-----+
| Ethernet Header |
+-----+
| IP Packet      |
+-----+
```

4. Send frame

write(raw_fd, frame, txlen)

Sends directly at Layer 2.

5. Debug logging

Extracts IP header:

```
struct iphdr *ip = (struct iphdr *)buf;
```

Prints:

TX IP src -> dst len=N

rx_loop()

```
static void rx_loop(...)
```

Purpose

Receives Ethernet frames and injects IP packets into TUN.

RX Processing Steps

1. Receive Ethernet frame

```
read(raw_fd, frame, sizeof(frame))
```

2. Validate EtherType

```
if (ntohs(eth->ether_type) != CUSTOM_ETHERTYPE)
    continue;
```

Filters unrelated traffic.

3. Remove Ethernet header

```
unsigned char *ip_pkt =
    frame + sizeof(struct ether_header);
```

4. Inject packet into TUN

```
write(tun_fd, ip_pkt, ip_len)
```

The kernel now treats this as an incoming IP packet.

5. Debug logging

Prints source/destination IPs.

main()

Usage

```
program tx <tun> <eth> <dst-mac>
program rx <tun> <eth> dummy
```

TX Example

```
./prog tx tun0 eth0 aa:bb:cc:dd:ee:ff
```

- Reads packets from tun0
 - Sends Ethernet frames through eth0
 - Uses specified destination MAC
-

RX Example

```
./prog rx tun0 eth0 dummy
```

- Receives custom EtherType frames from eth0
 - Injects packets into tun0
-

Architectural Notes

Why Use TUN?

TUN devices provide:

- virtual Layer-3 interfaces
- integration with Linux routing stack

This allows normal IP routing through the tunnel.

Why Raw Sockets?

Raw packet sockets allow:

- direct Ethernet frame access
- custom EtherType protocols

This bypasses TCP/UDP/IP transport layers.

Strengths

Simple Design

Very easy to understand:

- TUN ↔ Ethernet bridge

Efficient

Minimal encapsulation:

- only Ethernet header overhead

Good Learning Example

Demonstrates:

- TUN/TAP programming
- raw sockets
- Ethernet framing
- Linux networking internals

Weaknesses / Limitations

No Error Recovery

No:

- retransmission
- sequencing
- acknowledgements

Packet loss is unmanaged.

No MTU Handling

Large packets may:

- exceed Ethernet MTU
- fragment unexpectedly

No Validation

Assumes:

- packet contains valid IPv4
- lengths are sane

Could crash or misbehave with malformed traffic.

Unsafe Parsing

Example:

```
struct iphdr *ip = (struct iphdr *)buf;
```

No length checks before accessing headers.

Should verify:

```
n >= sizeof(struct iphdr)
```

Uses Blocking I/O

Both loops block indefinitely.

Potential improvements:

- `select()`
 - `poll()`
 - `epoll()`
-

No MAC Learning

Destination MAC must be manually provided.

No ARP or discovery mechanism exists.

No Security

Traffic is:

- unencrypted
- unauthenticated

Anyone on the LAN could inject fake frames.

Suggested Improvements

Add Length Validation

Example:

```
if (n < sizeof(struct ether_header))  
    continue;
```

Verify IPv4 Header

Check:

- version
 - IHL
 - total length
-

Use Nonblocking I/O

Would improve responsiveness and scalability.

Add Signal Handling

Graceful cleanup on:

- SIGINT
 - SIGTERM
-

Support Multiple Clients

Could maintain:

- MAC table
 - peer list
-

Add Encryption

Could encapsulate:

- AES-GCM
 - WireGuard-like framing
-

Add TAP Support

Using TAP instead of TUN would:

- carry full Ethernet frames
 - preserve ARP/broadcast traffic
-

Overall Assessment

This is a concise and educational example of:

- Linux TUN device programming

- raw Ethernet sockets
- custom EtherType protocols
- packet encapsulation

Conceptually, it behaves like a very primitive Layer-2 VPN or Ethernet tunnel.

The code is suitable for:

- experimentation
- labs
- protocol learning
- controlled environments

But not production-ready due to:

- lack of security
- insufficient validation
- no reliability handling
- blocking architecture.